

Message Queue + Visit Counter with Redis

Instructor: M.Sc. Abdelhakim Rashid

Group Members	Ahmad Ibrahim Elmufti – 4866
	Aya Khaled Abuoud – 4834
	Seraj Nouri AlFaitoury – 5102
Date	May 10, 2026
GitHub	github.com/ahm4dmufti/Cloude_Computing_Midterm_Project

1. Project Title and Description

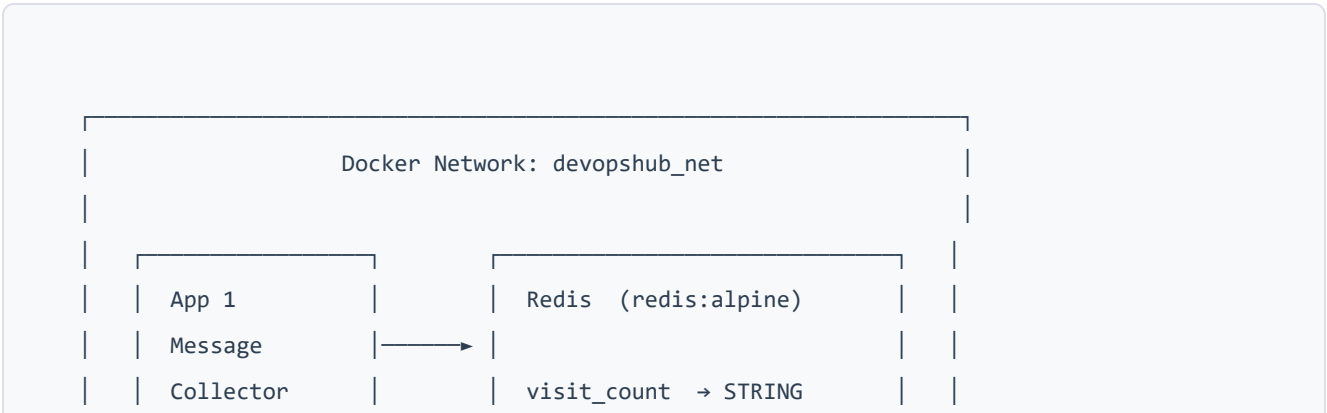
Title: DevOpsHub – Message Queue + Visit Counter with Redis

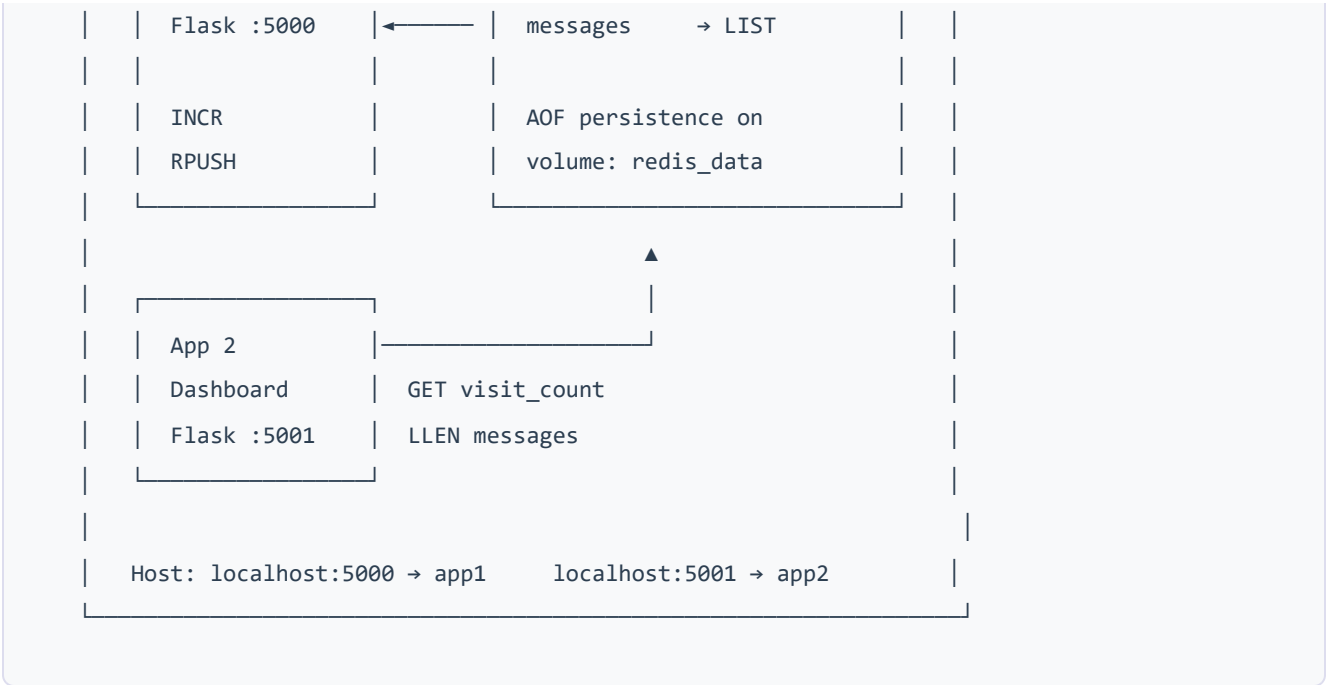
For this project we built a simple feedback system for a fictional startup called DevOpsHub. The idea is that users visit a page, submit a message, and everything gets stored in Redis. A separate dashboard page reads from the same Redis instance and shows how many messages were submitted and how many times the collector page was visited.

The whole thing runs as three Docker containers managed by Docker Compose — App 1 (the collector), App 2 (the dashboard), and Redis. All three are on the same Docker network so they can talk to each other by name without using IP addresses.

2. Architecture

Below is a diagram showing how the three containers are connected. App 1 writes to Redis, App 2 reads from Redis, and neither app talks directly to the other.





Service	Image	Port	Role
redis	redis:alpine	6379 (internal only)	Shared database
app1	project-app1	5000 → 5000	Collects messages, counts visits
app2	project-app2	5001 → 5001	Shows counts from Redis

3. How the Two Apps Interact with Redis

Both apps connect to Redis using `host='redis'` in the Python code. Docker's internal DNS resolves the name `redis` to the correct container automatically because all three services are on the same network.

What App 1 does

Every time someone opens the page, App 1 runs `INCR visit_count` which adds 1 to the counter and returns the new value. When the user submits a message, App 1 runs `R PUSH messages <text>` to add it to the end of a Redis list, then redirects back to the main page.

What App 2 does

App 2 only reads. Every page load runs `LLEN messages` to get the total number of items in the list, and `GET visit_count` to get the current visit number. It then displays both values on the page. App 2 never writes anything to Redis.

Redis Key	Type	Who writes	Who reads	Command
visit_count	String	App 1	App 2	INCR / GET
messages	List	App 1	App 2	R PUSH / LLEN

4. Prerequisites

To run this project you only need:

- **Docker Desktop** installed and running (it includes Docker Compose v2)
- Internet connection the first time, to pull the base images from Docker Hub
- Ports **5000** and **5001** free on your machine

No Python or Redis installation is needed on the host — everything runs inside the containers.

5. Step-by-Step Solution

Step 1 – Create the folder structure

Open a terminal and create the project folders:

```
mkdir project
cd project
mkdir app1
mkdir app2
```

The structure we are going to build:

```
project/
├── app1/
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
├── app2/
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
└── docker-compose.yml
```

Step 2 – Write App 1 (Message Collector)

Create `app1/requirements.txt` :

```
flask
redis
```

Create `app1/app.py` :

```
from flask import Flask, request, redirect, url_for
import redis

app = Flask(__name__)
r = redis.Redis(host='redis', port=6379, decode_responses=True)

HTML = """
<!DOCTYPE html>
<html>
<head>
  <title>Message Collector</title>
  <style>
    body {{ font-family: Arial, sans-serif; max-width: 600px; margin: 60px auto; background-color: #f0f0f0; }}
    .card {{ background: white; padding: 32px; border-radius: 10px; box-shadow: 0 2px 8px #ccc; }}
    h1 {{ color: #2c3e50; margin-bottom: 4px; }}
    .badge {{ display: inline-block; background: #3498db; color: white; padding: 4px 12px; border-radius: 20px; font-size: 14px; margin-bottom: 24px; }}
    textarea {{ width: 100%; padding: 10px; border: 1px solid #ccc; border-radius: 6px; font-size: 15px; box-sizing: border-box; }}
    button {{ margin-top: 12px; padding: 10px 28px; background: #2ecc71; color: white; border: none; border-radius: 6px; font-size: 15px; cursor: pointer; }}
  </style>
</head>
<body>
  <div class="card">
    <h1>DevOpsHub Feedback</h1>
    <span class="badge">Page visits: {visits}</span>
    <form method="POST" action="/submit">
      <textarea name="message" rows="4"
        placeholder="Enter your message, feedback or suggestion..." required></textarea>
      <br>
      <button type="submit">Submit Message</button>
    </form>
  </div>
</body>
</html>
"""

@app.route('/', methods=['GET'])
def index():
    visits = r.incr('visit_count')
    return HTML.format(visits=visits)
```

```
@app.route('/submit', methods=['POST'])
def submit():
    message = request.form.get('message', '').strip()
    if message:
        r.rpush('messages', message)
    return redirect(url_for('index'))

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Step 3 – Write App 2 (Dashboard)

Create `app2/requirements.txt`:

```
flask
redis
```

Create `app2/app.py`:

```
from flask import Flask
import redis

app = Flask(__name__)
r = redis.Redis(host='redis', port=6379, decode_responses=True)

HTML = """
<!DOCTYPE html>
<html>
<head>
    <title>DevOpsHub Dashboard</title>
    <style>
        body {{ font-family: Arial, sans-serif; max-width: 600px; margin: 60px auto; background-color: #f0f0f0; }}
        .card {{ background: white; padding: 32px; border-radius: 10px; box-shadow: 0 2px 8px #ccc; }}
        h1 {{ color: #2c3e50; margin-bottom: 24px; }}
        .stat {{ display: flex; justify-content: space-between; align-items: center; padding: 16px 0; border-bottom: 1px solid #eee; }}
        .stat:last-child {{ border-bottom: none; }}
        .label {{ font-size: 16px; color: #555; }}
        .value {{ font-size: 28px; font-weight: bold; color: #3498db; }}
        .footer {{ margin-top: 16px; font-size: 12px; color: #aaa; text-align: right; }}
    </style>
</head>
```

```
<body>
  <div class="card">
    <h1>Dashboard</h1>
    <div class="stat">
      <span class="label">Total Messages Collected</span>
      <span class="value">{messages}</span>
    </div>
    <div class="stat">
      <span class="label">Total Page Visits (App 1)</span>
      <span class="value">{visits}</span>
    </div>
    <p class="footer">Data refreshes on every page load</p>
  </div>
</body>
</html>
"""
```

```
@app.route('/')
def dashboard():
    messages = r.llen('messages')
    visits = r.get('visit_count') or 0
    return HTML.format(messages=messages, visits=visits)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5001)
```

Step 4 – Write the Dockerfile for App 1

Create `app1/Dockerfile` :

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000
```

```
CMD ["python", "app.py"]
```

Step 5 – Write the Dockerfile for App 2

Create `app2/Dockerfile` . It is the same as App 1 except the port:

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5001

CMD ["python", "app.py"]
```

Step 6 – Write docker-compose.yml

Create `docker-compose.yml` in the root of the project:

```
services:
  redis:
    image: redis:alpine
    container_name: redis
    volumes:
      - redis_data:/data
    command: redis-server --appendonly yes
    networks:
      - devopshub_net

  app1:
    build: ./app1
    container_name: app1
    ports:
      - "5000:5000"
    depends_on:
      - redis
```

```
networks:
  - devopshub_net

app2:
  build: ./app2
  container_name: app2
  ports:
    - "5001:5001"
  depends_on:
    - redis
  networks:
    - devopshub_net

volumes:
  redis_data:

networks:
  devopshub_net:
    driver: bridge
```

The `depends_on` makes sure Redis starts before the apps. The `devopshub_net` network lets all three containers resolve each other by name. The `redis_data` volume keeps the data even if the containers are stopped.

Step 7 – Build and run everything

From the `project/` folder run:

```
docker compose up --build -d
```

This builds the two app images, pulls Redis, creates the network and volume, and starts all three containers in the background. The first run takes a few minutes because it downloads the base images.

Step 8 – Check that all containers are up

```
docker compose ps
```

You should see all three containers with status **Up**:

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
app1	project-app1	"python app.py"	app1	20 seconds ago	Up 19 seconds	0.0.0.0:5
app2	project-app2	"python app.py"	app2	20 seconds ago	Up 19 seconds	0.0.0.0:5
redis	redis:alpine	"docker-entry..."	redis	21 seconds ago	Up 19 seconds	6379/tcp

Step 9 – Test the application

1. Open **http://localhost:5000** — the visit counter goes up by 1 on each load.
2. Type something in the form and click Submit Message.
3. Open **http://localhost:5001** — you will see the message count and visit count updated.

6. docker-compose.yml

```

services:
  redis:
    image: redis:alpine
    container_name: redis
    volumes:
      - redis_data:/data
    command: redis-server --appendonly yes
    networks:
      - devopshub_net

  app1:
    build: ./app1
    container_name: app1
    ports:
      - "5000:5000"
    depends_on:
      - redis
    networks:
      - devopshub_net

  app2:
    build: ./app2
    container_name: app2
    ports:
      - "5001:5001"
    depends_on:

```

```
- redis
networks:
  - devopshub_net

volumes:
  redis_data:

networks:
  devopshub_net:
    driver: bridge
```

7. Dockerfile – Web App 1

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

We use `python:3.11-slim` as the base image because it is small and has Python already installed. We copy `requirements.txt` first before the app code so that Docker can cache the pip install step — if we only change `app.py` later, it won't re-run pip. The app runs on port 5000.

8. Dockerfile – Web App 2

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .
```

```
EXPOSE 5001
```

```
CMD ["python", "app.py"]
```

Same structure as App 1. The only difference is the port — App 2 listens on **5001**.

9. Docker Images

After building, running `docker images` shows these images were created:

Repository	Tag	Image ID	Size
project-app1	latest	2943b5895b1e	216 MB
project-app2	latest	19cf64f0f072	216 MB
redis	alpine	d146f83b1e0f	134 MB

The two app images are both 216 MB because they are built on top of `python:3.11-slim` with Flask and the Redis client installed. The Redis image is much smaller because it uses the Alpine Linux base.

10. Access Instructions

Once the containers are running, open a browser and go to:

App	URL	What you will see
App 1 – Message Collector	<code>http://localhost:5000</code>	A form to submit messages with a visit counter at the top
App 2 – Dashboard	<code>http://localhost:5001</code>	Total messages collected and total page visits

To stop the containers:

```
docker compose down
```

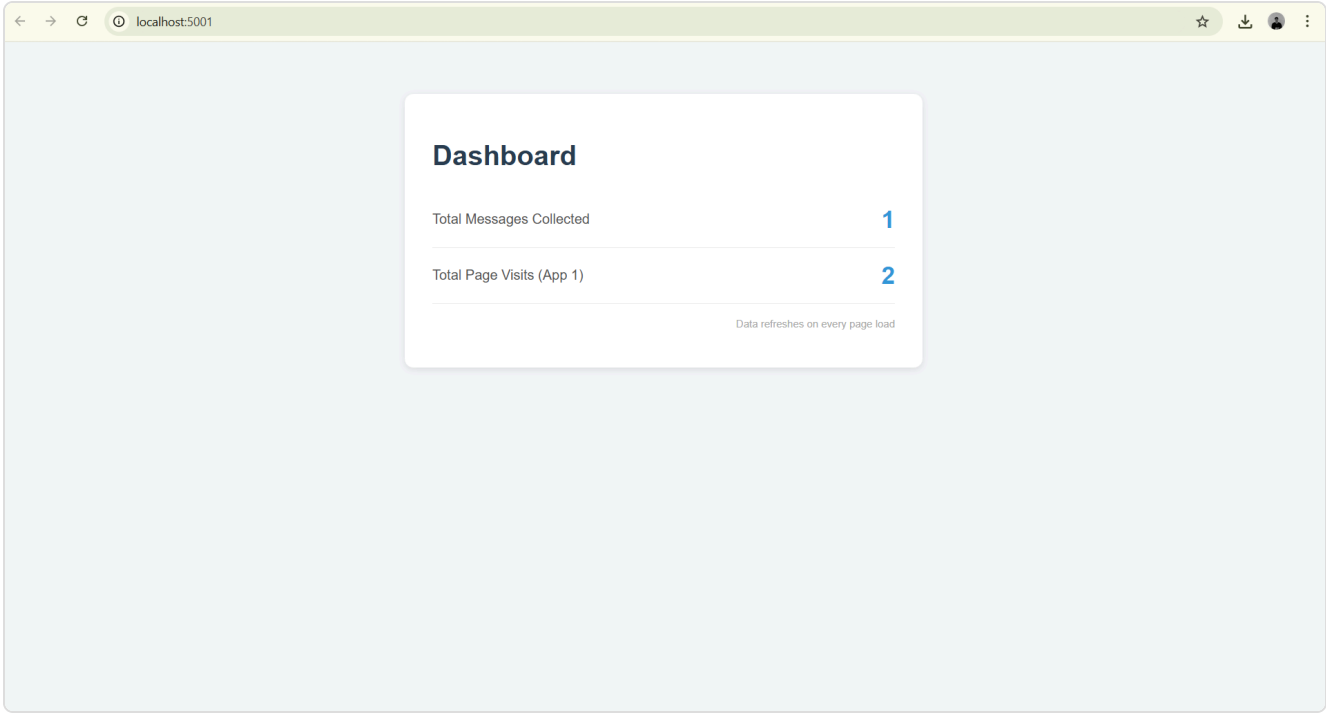
11. Screenshots

Screenshot 1 – docker compose ps showing all three services Up

```
PS C:\Users\ahmad\Desktop\project> docker compose ps
NAME          IMAGE          COMMAND                  SERVICE  CREATED      STATUS      PORTS
app1          project-app1   "python app.py"         app1     18 minutes ago Up 18 minutes 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
app2          project-app2   "python app.py"         app2     18 minutes ago Up 18 minutes 0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp
redis         redis:alpine   "docker-entrypoint.s..." redis    18 minutes ago Up 18 minutes 6379/tcp
PS C:\Users\ahmad\Desktop\project>
```

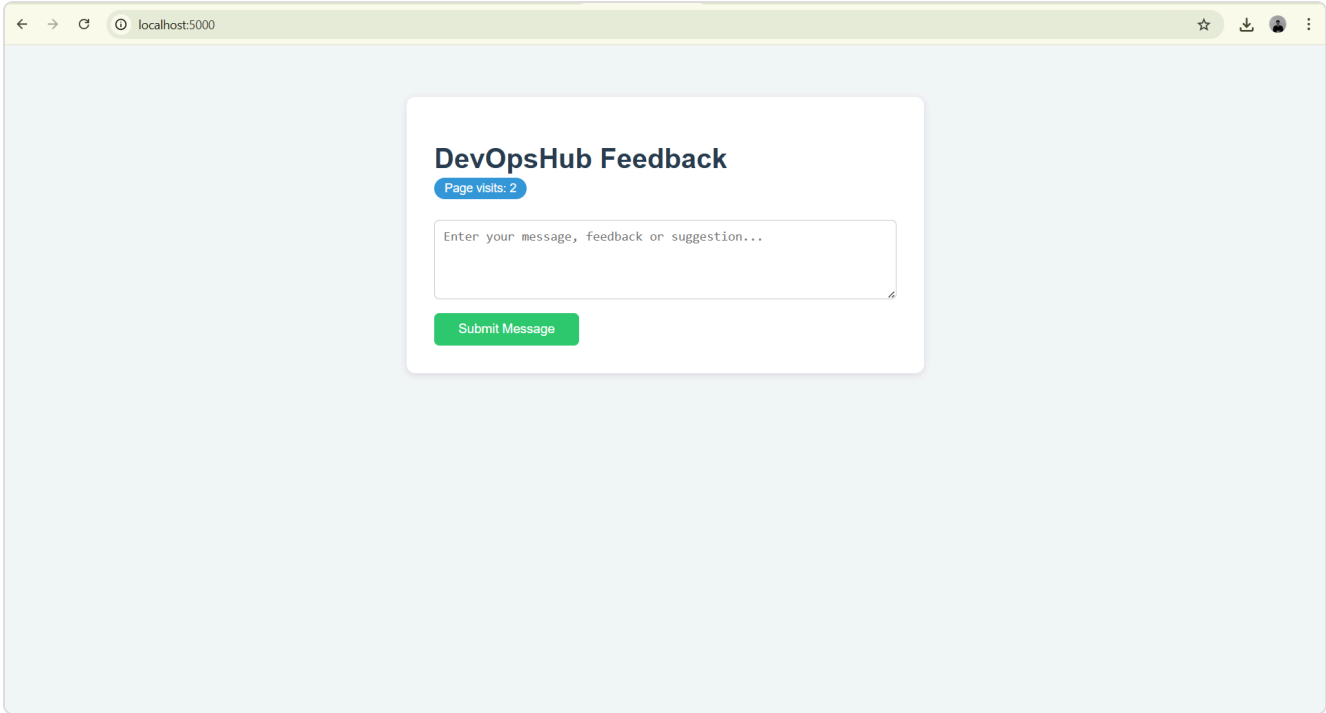
All three containers running: app1 (port 5000), app2 (port 5001), redis (port 6379)

Screenshot 2 – Browser showing App 1 (Message Collector) at localhost:5000



App 1 running at http://localhost:5000 — feedback form with visit counter badge

Screenshot 3 – Browser showing App 2 (Dashboard) at localhost:5001



App 2 running at <http://localhost:5001> — total messages and visit count pulled from Redis